

CS 546

Computer System Security

Lecture 4: Crypto (Integrity - I)

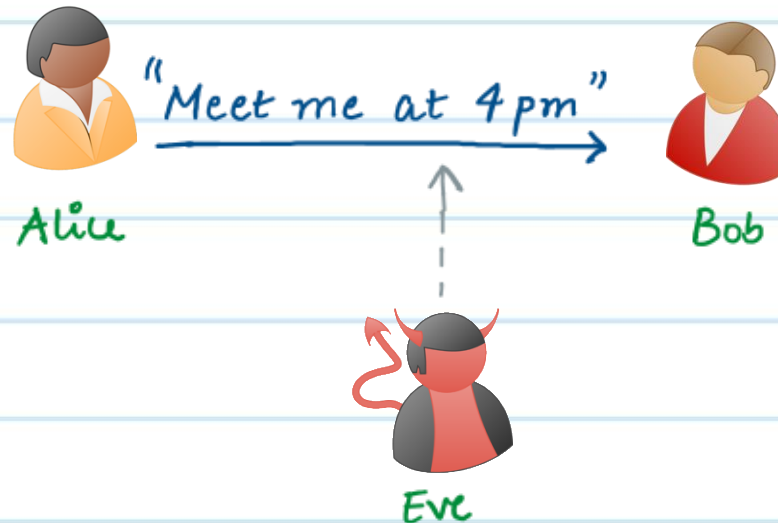
Instructor: Aarushi Goel

Fall 2026

Message Authentication Codes (MACs)

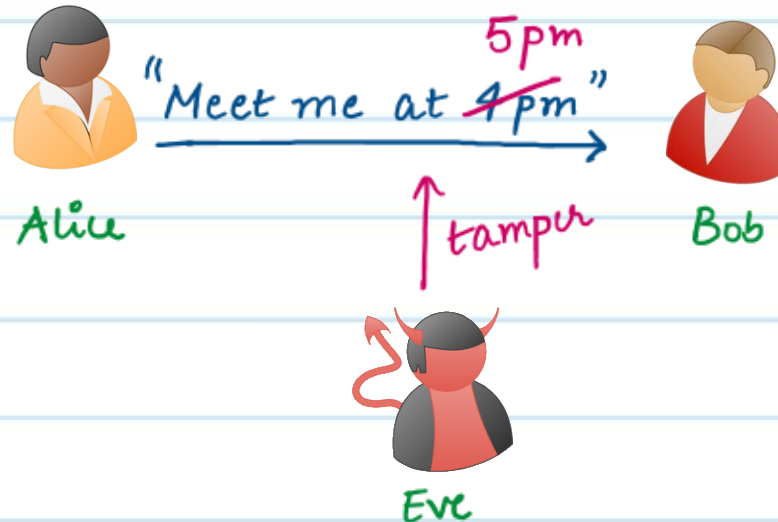
Private Communication

- * So far we have considered the problem of private communication, where Alice wants to send a secret message to Bob, while making sure **Eve** who might be eavesdropping on their communication.
- * Encryption helps with this.
- * But an Eve who is only listening to a private communication, is still a pretty "well-behaved" adversary.



Tamper-Proof Communication

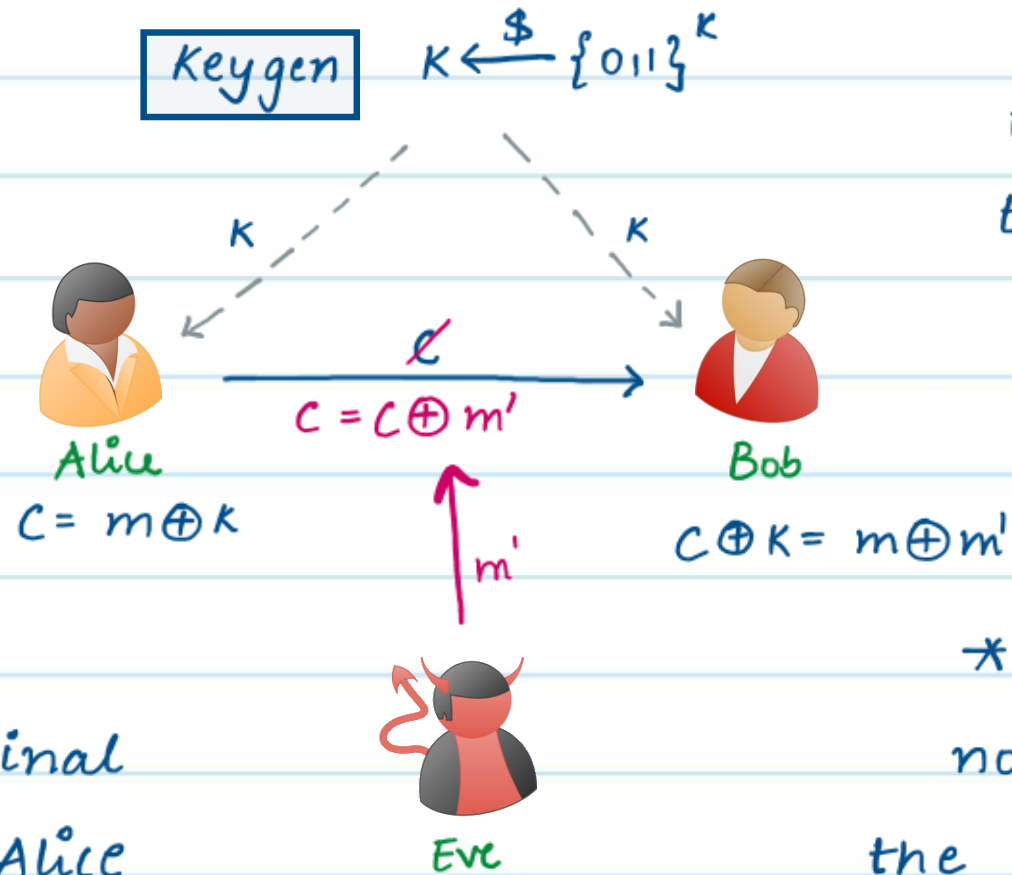
- * In reality, adversaries could be much worse.
- * What if the adversary can not only eavesdrop on a communication channel, but also *tamp* with messages on this communication channel.
- * Does encryption naturally help prevent this? **No!**



Q Why doesn't one-time pad encryption make the underlying message tamper-proof?

* Eve can change the ciphertext $c \rightarrow c \oplus m'$ for any m' of its choice.

* Bob then ends up decrypting $m \oplus m'$ instead of the original message m that Alice intended to send.



* Bob has no way to tell whether this is the original message or tampered message.

* Note that Eve does not even need to know the key or message m to launch this attack.

Message Integrity in the Non-Digital World

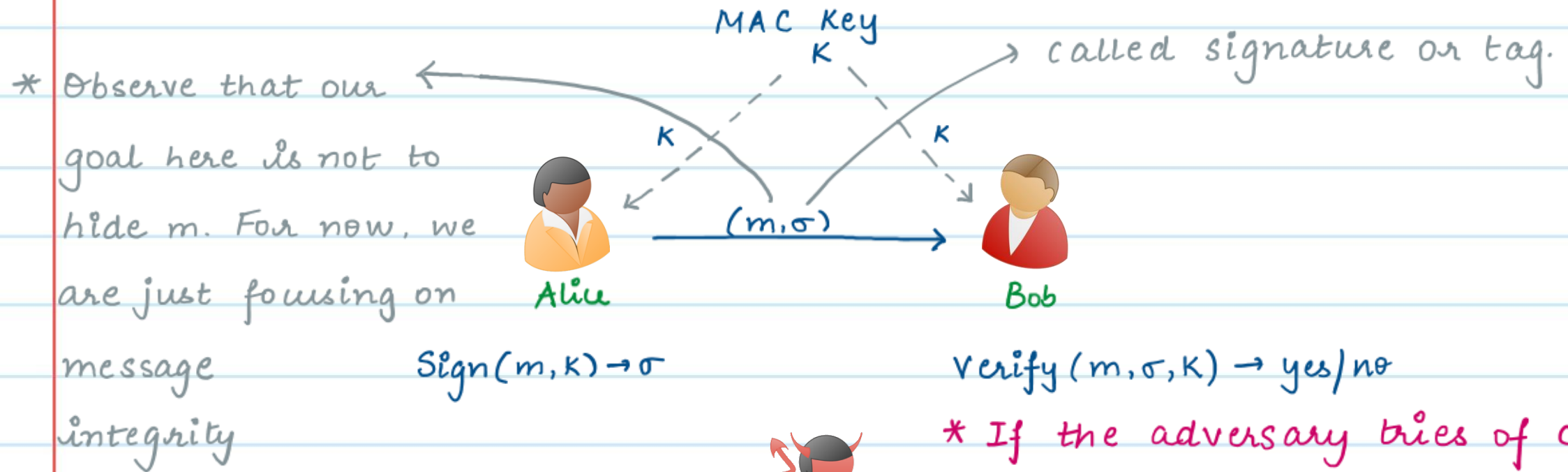
- * In the physical world, we use signatures on documents to ensure unauthorised entities cannot tamper with them, without forging the signature, which is hard to do.
- * We need something similar in the digital world.



Message Authentication Codes

- * Digital Analogue of Physical Signatures.
- * Alice (signer) signs a message m to produce a signature σ .
- * Bob (verifier) can verify that σ is indeed generated for m .
- * We will assume Alice and Bob share a key that is used for signing and verification.
- * Adversary does not have this key.
- * It should be impossible (or very hard) for an adversary to forge a valid signature with respect to this key on any other message.

Message Authentication Codes (MACs)



Adversary

- * We will later see how to combine Encryption and MACs to ensure integrity of encrypted messages.
- * If the adversary tries to change $m \rightarrow m'$, they will also have to find a corresponding σ' , such that $\text{Verify}(m', \sigma', K) \rightarrow \text{yes}$. This should be hard if they don't have K .

Message Authentication Codes (MACs)

Defined by three algorithms:

- * $\text{Keygen} \xrightarrow{\$} K$
- * $\text{Sign}(K, m) \rightarrow \sigma$
- * $\text{Verify}(K, m, \sigma) \rightarrow \text{yes / no}$

→ **Correctness:** Let $\text{Keygen} \xrightarrow{\$} K$, then $\forall m$
 $\Pr[\text{Verify}(K, \text{Sign}(K, m), m)] = 1$

→ **Unforgeability:** A PPT adversary who does not have the key K , cannot forge a valid MAC on any message of its choice, even if it sees a MAC on some message.

Constructing * One-time * MACs.

→ the key can only be used once

Let $M = \mathbb{Z}_p$, $K = \mathbb{Z}_p \times \mathbb{Z}_p$, $S = \mathbb{Z}_p$. Recall $(\mathbb{Z}_p, + \bmod p, \times \bmod p)$ is a field.
 $\downarrow$
 $\{0, 1, \dots, p-1\}$, p is prime

* Keygen: Sample $K_1 \xleftarrow{\$} \mathbb{Z}_p$ and $K_2 \xleftarrow{\$} \mathbb{Z}_p$. Set $K = (K_1, K_2)$

* Sign(m, K): Parse $K = (K_1, K_2)$.

$$\sigma = (K_1 \cdot m + K_2) \bmod p.$$

* Verify(m, σ, K): Parse $K = (K_1, K_2)$

$$b = \begin{cases} 1, & \text{if } (K_1 \cdot m + K_2) \bmod p = \sigma \\ 0, & \text{otherwise} \end{cases}$$

Q Does this scheme satisfy correctness & unforgeability?

* **Correctness:** We need to show that $\forall m \in \mathbb{Z}_p, \forall K \in \mathbb{Z}_p \times \mathbb{Z}_p,$
 $\Pr[\text{Verify}(m, \text{Sign}(m, K), K) = 1] = 1$

Proof:

$$\begin{aligned} & \text{Verify}(m, \text{Sign}(m, K), K) \\ &= \text{Verify}(m, \text{Sign}(m, (K_1, K_2)), (K_1, K_2)) \\ &= \text{Verify}(m, K_1 \cdot m + K_2 \bmod p, (K_1, K_2)) \\ &= 1. \end{aligned}$$

This holds with probability 1.

* **Unforgeability:** We need to show that given a MAC on some message, the adversary cannot forge another MAC on some other message of its choice.

Proof: The adversary first receives σ on a message m of its choice.
where $\sigma = (K_1 \cdot m + K_2) \bmod p$.

To win, it must send a valid σ' on another message $m' \neq m$.

$$\Rightarrow \sigma' = (K_1 \cdot m' + K_2) \bmod p \quad (\text{if } \sigma' \text{ is valid})$$

$$\Rightarrow K_2 = (K_1 \cdot m' - \sigma') \bmod p \quad (\text{we know that } K_2 = (K_1 \cdot m - \sigma) \bmod p)$$

$$\Rightarrow (K_1 \cdot m - \sigma) \bmod p = (K_1 \cdot m' - \sigma') \bmod p$$

$$\Rightarrow K_1 (m - m') = (\sigma - \sigma') \bmod p$$

$$\Rightarrow K_1 = \frac{(\sigma - \sigma')}{(m - m')} \bmod p$$

But since K_1 was chosen at random,

$$\Pr \left[K_1 = \frac{(\sigma - \sigma')}{(m - m')} \bmod p \right] = \frac{1}{p}$$

In other words, the probability that adversary can find m, m' such that $K_i = \frac{(\sigma - \sigma') \bmod p}{(m - m')}$ is $\frac{1}{p}$.

- * If p is very large (eg. 2^{128}), then this probability is very small (or negligible) and we can safely assume that **no** adv will be able to win except with negligible probability.

Should this be "no" or
"no PPT"?

- * But what if p is not large enough, i.e., our messages come from a small space?

Constructing One-time MACs for Small Message Spaces

Let $M = \mathbb{Z}_p$. Let $n = 128$. $K = \mathbb{Z}_p^n \times \mathbb{Z}_p^n$, $S = \mathbb{Z}_p^n$

* Keygen: $\forall i \in [n]$, sample $K_1^i \xleftarrow{\$} \mathbb{Z}_p$ and $K_2^i \xleftarrow{\$} \mathbb{Z}_p$. Set $K = (\{K_1^i\}_{i \in [n]}, \{K_2^i\}_{i \in [n]})$

* Sign(m, K): $\forall i \in [n]$: $\sigma_i = (K_1^i \cdot m + K_2^i) \bmod p$.

Set $\sigma = (\sigma_1, \dots, \sigma_n)$

* Verify(m, σ, K): $\forall i \in [n]$:

$$b_i = \begin{cases} 1, & \text{if } (K_1^i \cdot m + K_2^i) \bmod p = \sigma_i \\ 0, & \text{otherwise} \end{cases}$$

Set $b = b_1 \wedge b_2 \wedge \dots \wedge b_n$

↓
AND

Q Is this scheme unforgeable? Why?

MACing Many Messages using the same Key

Q Can we use these schemes to sign more than one message using the same key?

A No. If the adversary sees two signatures in this construction, they will be able to recover the key.

eg Let's assume they see (m_1, σ_1) & (m_2, σ_2) .

where $\sigma_1 = (K_1 \cdot m_1 + K_2) \bmod p$ & $\sigma_2 = (K_1 \cdot m_2 + K_2) \bmod p$.

* The adv can compute $K_1 = \left(\frac{\sigma_2 - \sigma_1}{m_2 - m_1} \right) \bmod p$ and then compute $K_2 = (\sigma_1 - K_1 \cdot m_1) \bmod p$.

* Once it recovers (K_1, K_2) , it can use this key to forge signatures on any messages of its choice.

MACing Many Messages using the same Key

(Many time)

- * Message Authentication Codes that allow the key to be reused for safely signing many messages are defined exactly as one-time MACs, except that now we say that even if the adversary sees MACs on many messages of his choice, they still cannot forge a valid MAC on some new message.

Construction of Many-time MAC Scheme

Let $\{f_k\}$ be a family of PRFs.

- * Keygen: Sample a PRF Key K .
- * Sign(m, K): $\sigma = f_K(m)$
- * Verify(m, K):
$$b = \begin{cases} 1, & \text{if } \sigma = f_K(m) \\ 0, & \text{otherwise} \end{cases}$$

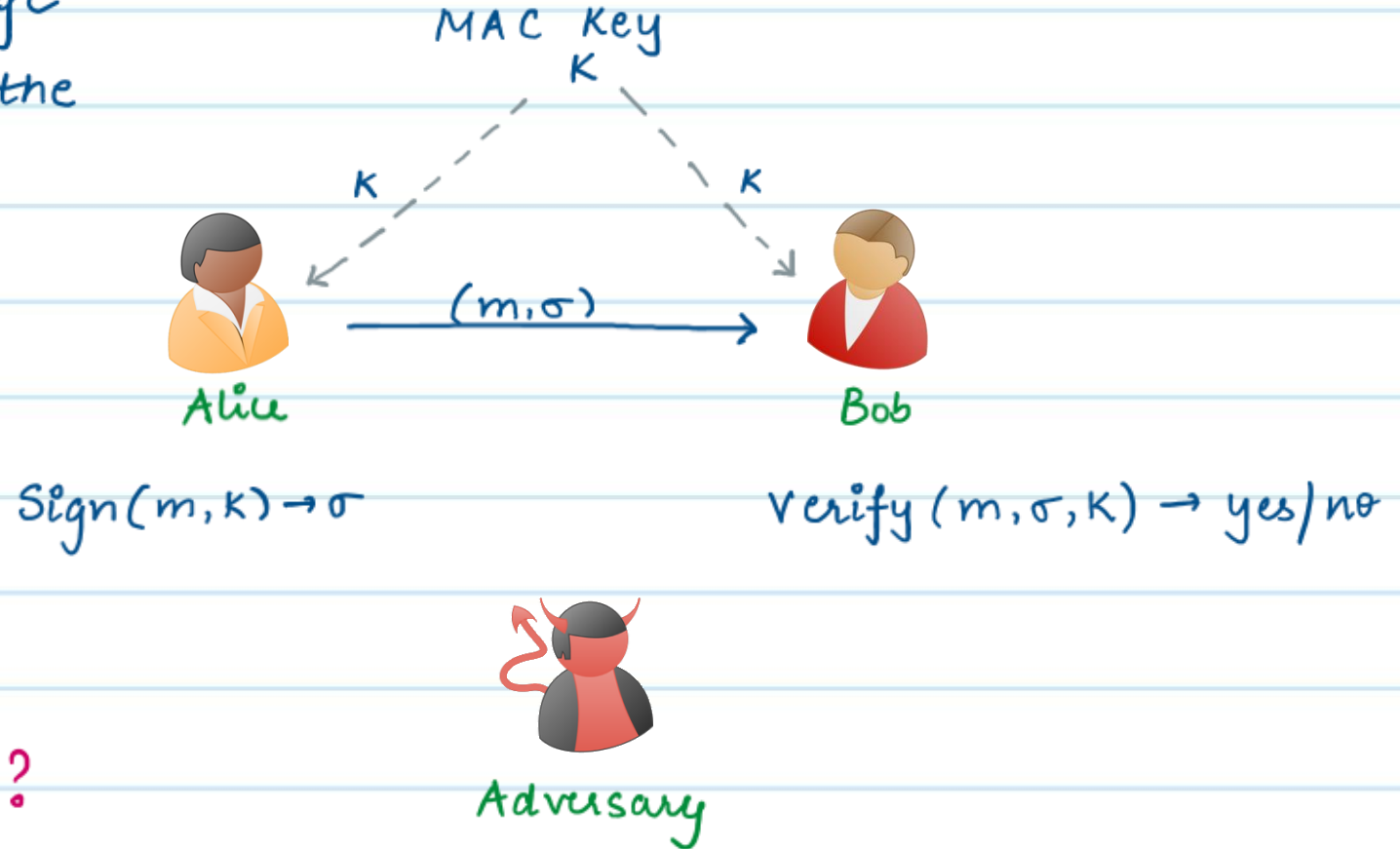
* Correctness: straightforward

* Unforgeability: An adversary who can break unforgeability of this scheme should be able to distinguish between the outputs of this PRF and the outputs of a random function. But we know that no PPT adversary can do that. Hence this scheme is unforgeable.

Message Authentication Codes (MACs)

- * MACs only help with message integrity, they don't hide the message
- * Encryption only hides the message, doesn't help with message integrity.

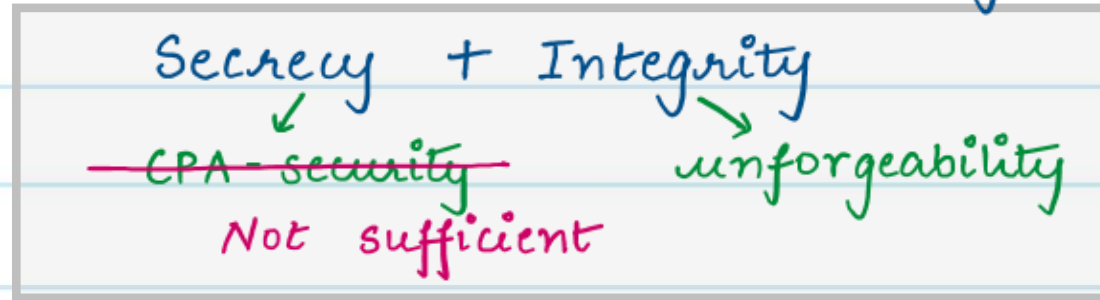
Q Can we somehow combine encryption and MACs to get both privacy and integrity?



Authenticated Encryption

Authenticated Encryption

- * An encryption scheme that can both hide the message and cannot be tampered with.



- Recall that we came up with the definition of CPA-security to only account for adversaries who can only eavesdrop.
- Now we are dealing with stronger adversaries who can modify/maul the ciphertexts.
- Such adversaries might be able to violate privacy of CPA-secure encryption
eg padding oracle attacks

If an adversary is able to modify the ciphertexts and learn whether the receiver (Bob) was able to decrypt the modified ciphertexts or not, they may be able to recover the original message. — This is the main idea behind padding oracle attacks

- * Therefore CPA-security is not sufficient for guaranteeing secrecy in authenticated encryption.
- * We need a stronger notion of secrecy — CCA-security
(chosen ciphertext attack)

Authenticated :
Encryption

Secrecy + Integrity
CCA-security unforgeability

Chosen-Ciphertext - Attack (CCA - Security)

- * Recall that in CPA-security (the simpler definition of privacy that we discussed for encryption schemes), we said that any PPT adversary who sees many encrypted messages, can still not distinguish between encryptions of two messages of its choice.
- * In **CCA-security**, we give the adversary additional power.
- * It can now not only see encrypted messages but also the corresponding decrypted messages. They can still not distinguish between encryptions of two messages of its choice.
 - they obviously don't get decryptions of these "challenge" ciphertexts.
- * It's like the adversary has a decryption oracle for the other ciphertexts

Chosen-Ciphertext - Attack (CCA - Security)

- * The adversary can use this decryption oracle to decrypt any polynomial number of ciphertexts of its choice, except the challenge ciphertext itself.
- * CCA security intuitively captures **non-malleability**, i.e., if the adversary tries to modify a given ciphertext, the result is either an **invalid ciphertext** or one that decrypts to a plaintext having no relation to the original one.

Q Does this realistically model any real-world attack? Do real-world adversaries really have such decryption oracles?

1. Depending on the application, they might.
2. Or even if they don't, they might be able to "influence" what gets decrypted and try to learn partial information from that.

example 1: padding oracle attacks

example 2: Country A intercepts a conversation between an enemy country B and their armed forces. Country A modifies the encrypted message and tries to deduce the original message based on their behavior.

* CCA-security is the strongest definition to ensure privacy in all such scenarios.

Authenticated Encryption

Definition: An authenticated encryption scheme must be CCA-secure and unforgeable. $\rightarrow$ defined similarly to MACs.

Construction: CPA secure encryption + CMA-secure MAC
 $\downarrow$
Authenticated Encryption
(CCA-secure & unforgeable)

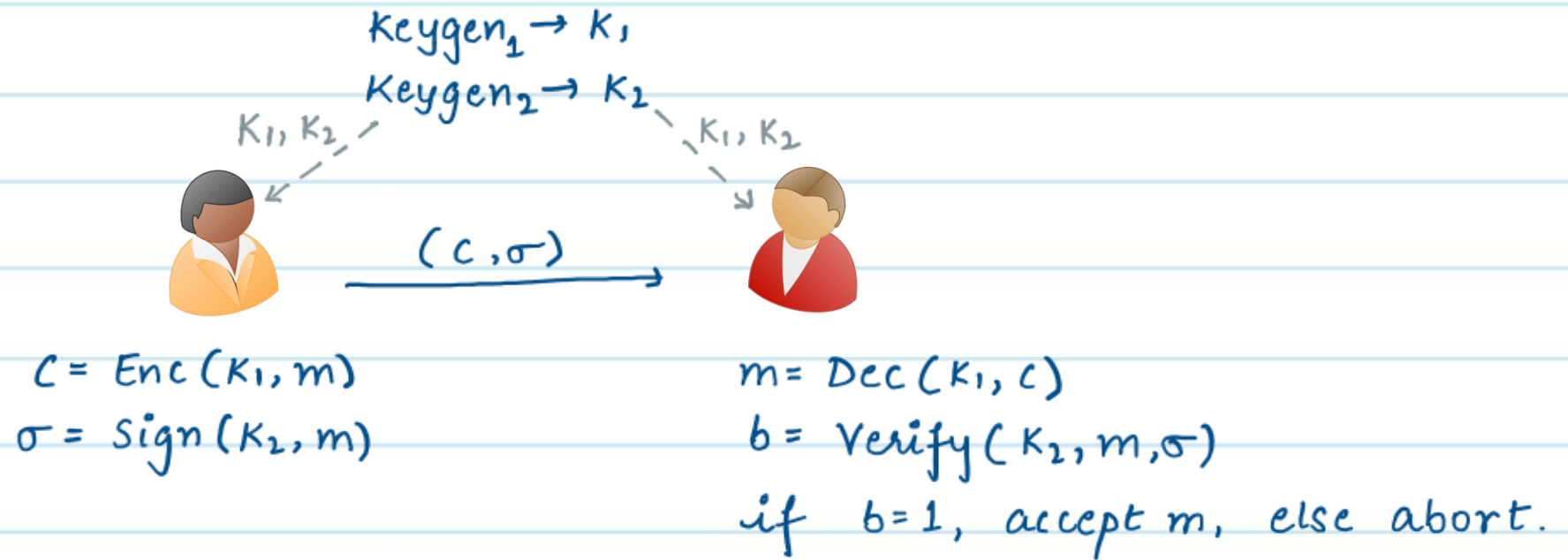
Q How should we combine CPA-secure encryption & CMA-secure MACs?

Attempt 1:

Encrypt - and - MAC

Let $(\text{Keygen}_1, \text{Enc}, \text{Dec})$ be a CPA-secure Encryption Scheme.

Let $(\text{Keygen}_2, \text{Sign}, \text{Verify})$ be a many-time MAC Scheme



Q Is this a good strategy for designing an authenticated encryption scheme? **No!**

Why?

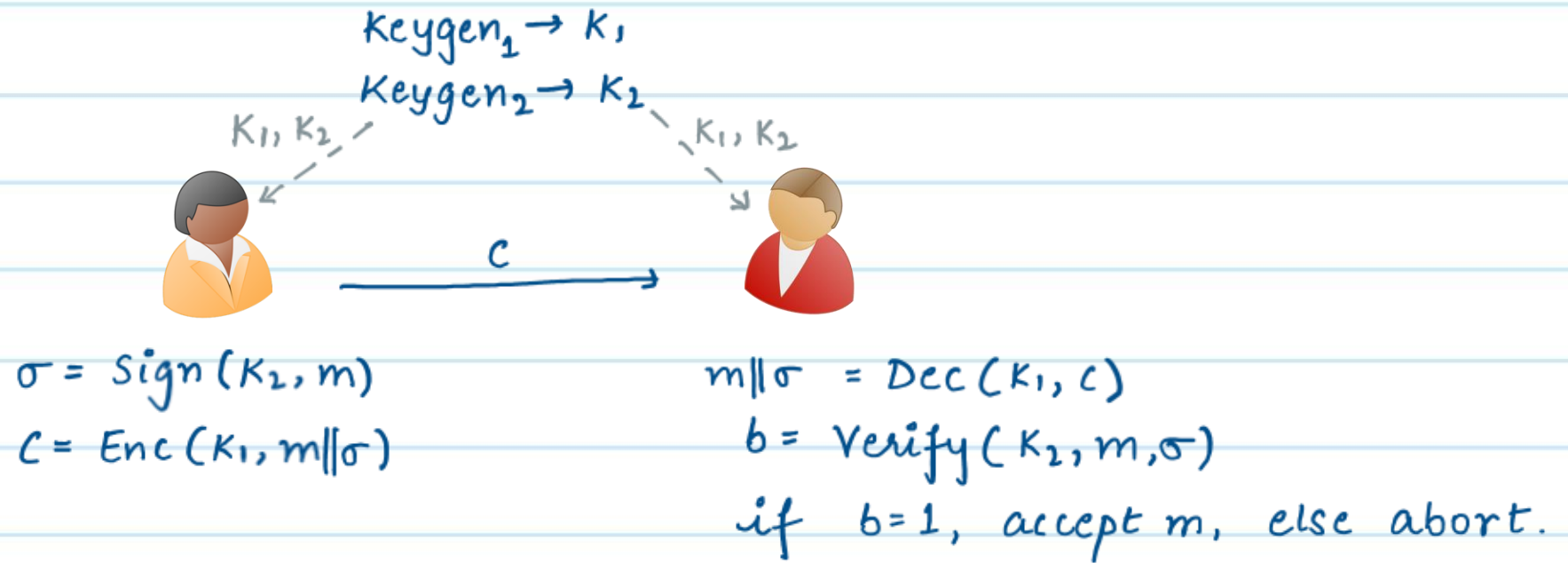
- * σ doesn't even try to hide the message.
- * It's possible that the MAC scheme reveals the message in the clear.

Attempt 2:

MAC - then - Encrypt

Let $(\text{Keygen}_1, \text{Enc}, \text{Dec})$ be a CPA-secure Encryption Scheme.

Let $(\text{Keygen}_2, \text{Sign}, \text{Verify})$ be a many-time MAC Scheme



Q Is this a good strategy for designing an authenticated encryption scheme? **No!**

Why?

- * c is only a CPA-secure encryption of $m||\sigma$. It may be susceptible to chosen-ciphertext attacks.
- * Eg. There are two-sources of decryption failure here:
 1. c itself may be an ill-formed ciphertext
 2. σ may not be valid MAC for m .

Consider an adversary who can distinguish between these two error messages can launch a padding oracle style attack.

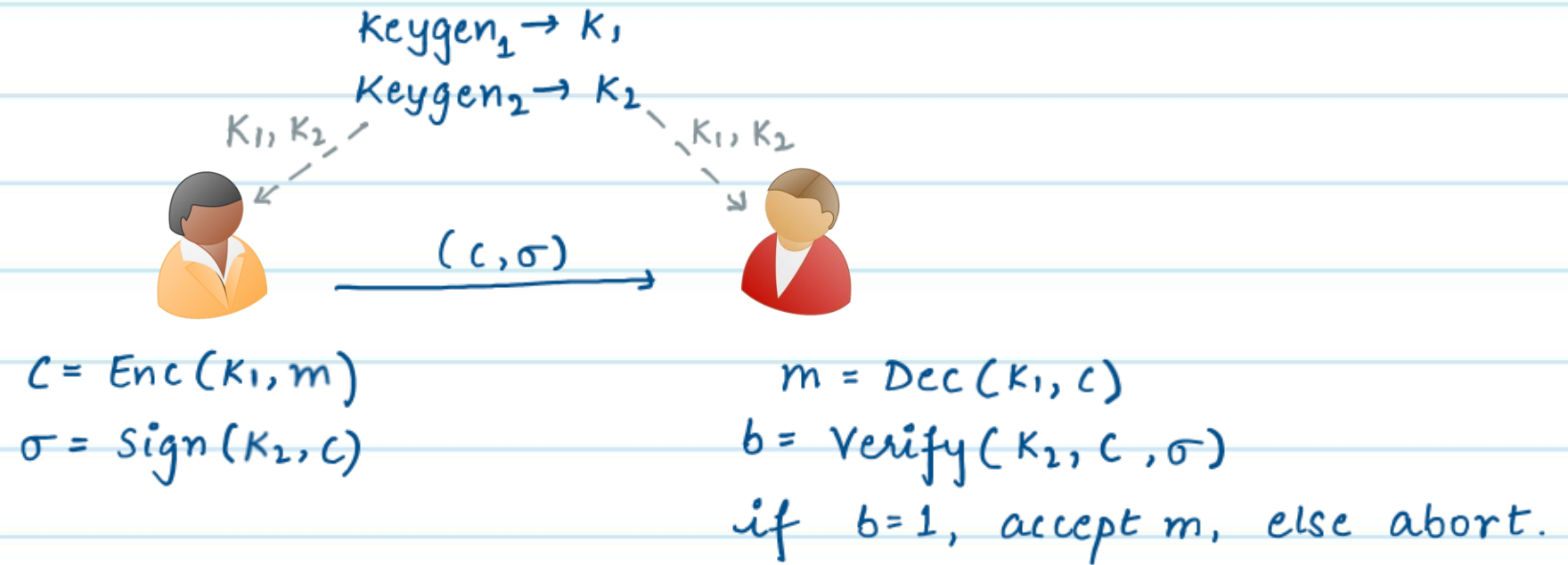
Such attack was successfully carried out on configurations of IPsec that used MAC-then-encrypt.

Attempt 3:

Encrypt-then-MAC

Let $(\text{Keygen}_1, \text{Enc}, \text{Dec})$ be a CPA-secure Encryption Scheme.

Let $(\text{Keygen}_2, \text{Sign}, \text{Verify})$ be a many-time MAC Scheme

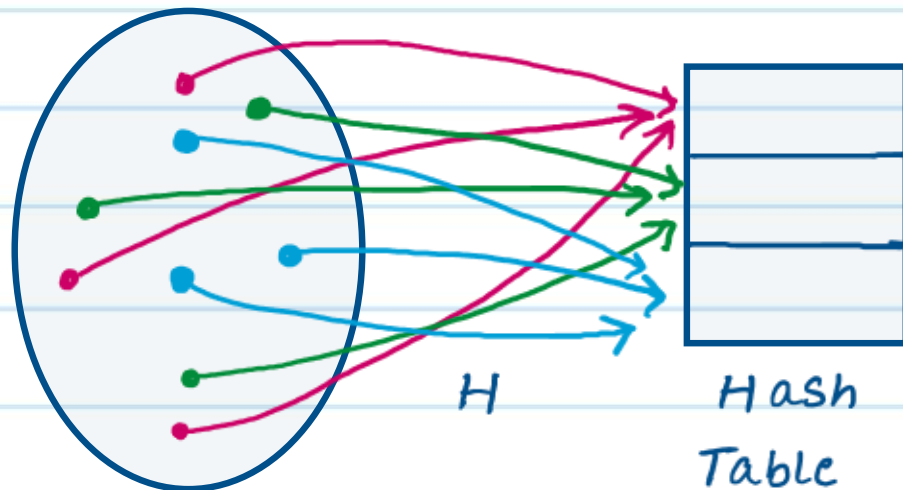


Q Is this a good strategy for designing an authenticated encryption scheme? Yes!

Cryptographic Hash Functions

Hash Functions

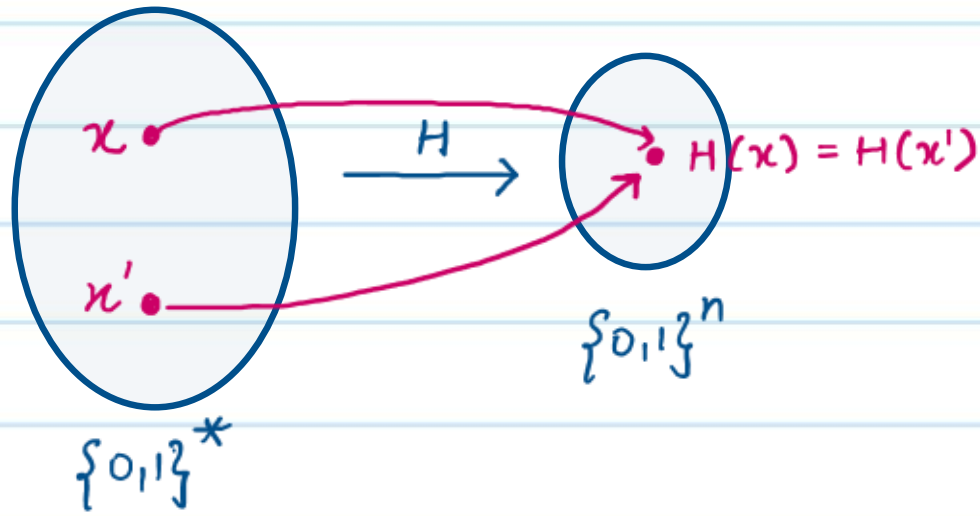
- * Functions that take inputs of arbitrary length and compress them into short, fixed length outputs.
- * $H: \{0,1\}^* \rightarrow \{0,1\}^n$
 ↓
 arbitrary length
- * Traditional use of hash functions was in data structures to enable faster lookup time when storing a set of elements.



To retrieve an element x it suffices to compute $H(x)$ and probe that row in the hash table to look for x . This is more efficient than looking up x in the original list.

Collisions in Hash Functions

- * A hash function naturally has *collisions*, where a collision is a pair of distinct elements x and x' , such that $H(x) = H(x')$.
- * A good hash function yields few collisions, because when a collision happens, two elements end up in the same row in the hash table, increasing the lookup time.



Collision - Resistant Hash Functions

- * Collision-resistant hash functions (CRHF) are cryptographic versions of hash functions, where the goal is not just to minimize the number of collisions, but it should also be infeasible for any probabilistic polynomial-time algorithm to find a collision in H .
- * Note that CRHFs are still compression functions, so collisions exist, but we just want such collisions to be hard to find.
- * We typically consider Keyed CRHFs.

$$H_K(x) = H(K, x)$$

unlike all other primitives we have discussed so far, here we don't care about hiding the key.

- * It should be hard to find a collision in H^K , for randomly chosen K , even given K .

Hash Functions: Theory vs Practice

Theoretical Constructions

$$\{H_k\}_{k \in \{0,1\}^n}$$

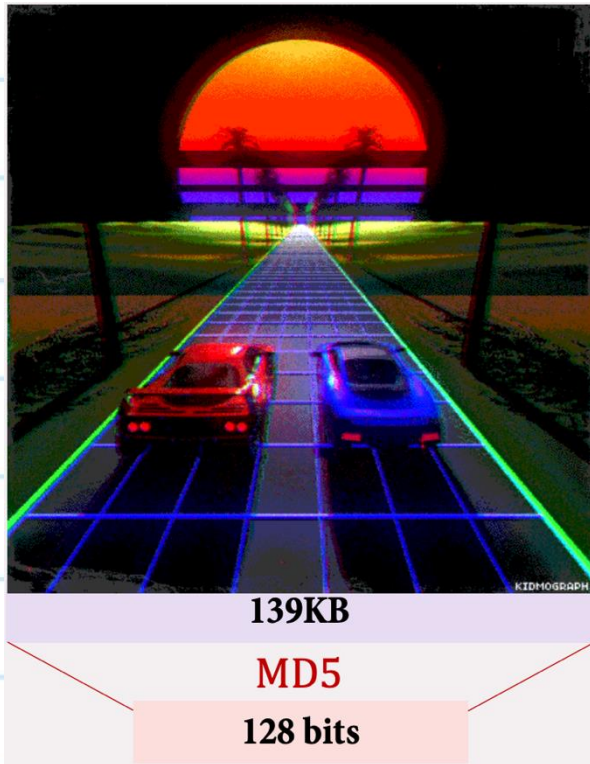
- * typically keyed functions
- * there is a security parameter n .
- * Security proved under well-studied computational hardness assumptions

Practical Constructions

eg SHA3, SHA256, ~~SHA1~~, ~~MD5~~, ~~MD4~~ etc
broken

- * unkeyed, so single functions, not a family of functions
- * No security parameter knob
- * Use until someone finds a collision.

Example: Collisions in MD5



5c827c0eba9cfaa647c1a489bea77c60



5c827c0eba9cfaa647c1a489bea77c60

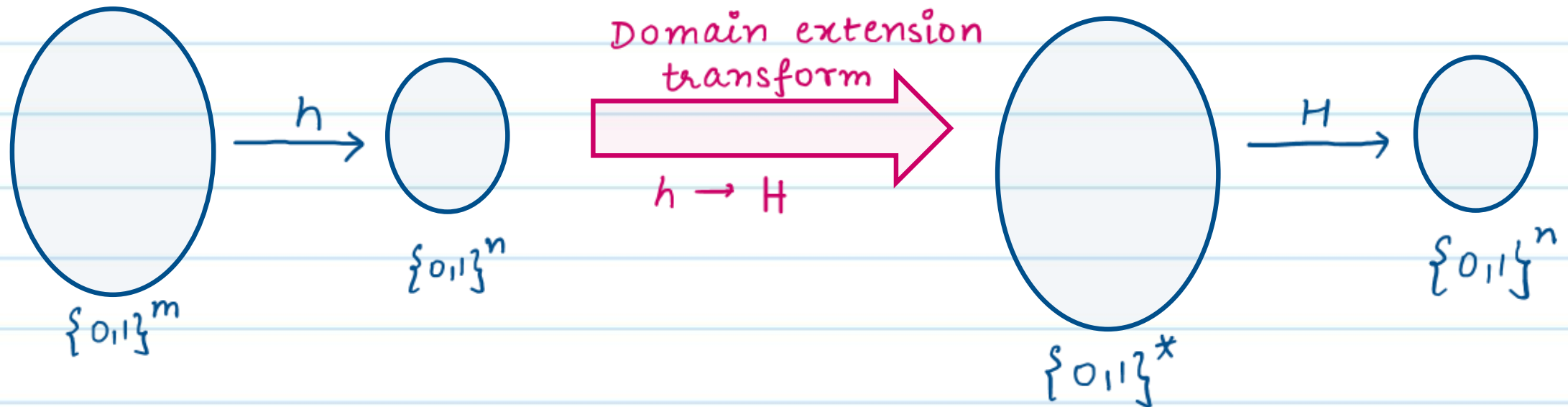
Generic Attacks on Hash Functions

- * Let $H: \{0,1\}^* \rightarrow \{0,1\}^n$ (we drop the key here, and assume it is implicit).
- * **Trivial Attack for finding collisions:** Evaluate H on 2^n+1 distinct inputs. By pigeonhole principle, two of the outputs must be equal. This attack takes $O(2^n)$ time. Can we do better?
- * **Birthday Attack:** If there are q students in class and assuming each student's birthday is a random $y_i \xleftarrow{\$} [365]$. What is the probability of collision?
 - $q = 366 \Rightarrow \text{prob} = 1$
 - $q = 23 \Rightarrow \text{prob} = 1/2$
 - $q = 70 \Rightarrow \text{prob} = 99.9\%$
- In general, birthday analysis shows that for $y_1, \dots, y_q$ chosen uniformly from $[N]$, the probability of collision is approx $1/2$, when $q \approx \sqrt{N}$.

- * In our setting, this means that when the hash function output is on length n (i.e., the range is of size $N=2^n$), if we evaluate on $q=2^{n/2}$ distinct inputs, we will see a collision with probability $1/2$.
- * From concrete efficiency perspective, for a hash function to resist collision-finding attacks that run in time T , the output length of the hash function must be at least $2\log T$ bits (i.e., range is of size $2^{2\log T} = T^2$).
- * For $T=2^{128}$, we need output length of hash function to be at least 256 bits.

Domain Extension

- * Hash functions are often constructed by first designing a CRHF that handles fixed-length inputs, and then using domain extension to handle arbitrary-length inputs.



- * Merkle-Damgård transform is an example of domain extension.